



6



11



1650 sp

عملي

محتوى مجاني غير مخصص للبيع التجاري

RB Informatics; 2025/11/25

كلية الهندسة المعلوماتية

السنة الرابعة

LOOPS AND CASE STATEMENT

عملي مشترك



نظم التشغيل

في المحاضرة السابقة تعرفنا على المتغيرات والتوابع في ال Shell Scripting وفي هذه المحاضرة سنتعرف على الحلقات التكرارية و الشروط .

الحلقات التكرارية

حلقة For :

تستخدم للتكرار عبر مجموعة من القيم أو الكلمات داخل سلسلة نصية.

```
for word in A B C; do
    echo $word
done
```

```
for word in A B C
do
    echo $word
done
```

حلقة While :

1. تنفذ الكود طالما أن الشرط صحيح (true) ويتوقف التنفيذ عندما يصبح الشرط false .
2. يمكن إيقافها مبكراً باستخدام الأمر break.

```
while [ condition ]; do
    command
done
```

```
while [ condition ]
do
    command
done
```

حلقة Until :

1. تنفذ الكود طالما أن شرط التحكم غير محقق (false) ويتوقف التنفيذ عندما يصبح الشرط true .
2. تعد عكس حلقة while تماماً.

```
until [ condition ]; do
    command
done
```

```
until [ condition ]
do
    command
done
```

مثال تطبيقي :

لدينا ملف باسم users.txt يحتوي على أسماء المستخدمين وكلمات المرور الخاصة بكل مستخدم، بحيث يكون كل سطر بالشكل :

username:password

```
testuser:0123456789
user1:0123456789
user2:0000
```

يقوم السكريبت بما يلي:

- يتحقق من وجود اسم مستخدم مُدخل من قبل المستخدم.
- إذا كان المستخدم موجوداً، يسمح له بثلاث محاولات فقط لإدخال كلمة المرور.
- في حال فشل جميع المحاولات، يتوقف السكريبت.
- في حال نجاح تسجيل الدخول، ينتقل إلى مجلد المستخدم الأساسي ويعرض الملفات الموجودة فيه.

```
#!/bin/bash
# Path to password file
PASS_FILE="./users.txt"
# Prompt for username
read -p "Enter username: " user

# Check if user exists in the file
if ! grep -q "^$user:" "$PASS_FILE"; then
    echo "❌ User '$user' not found in database."
    exit 1
fi

attempt=1
# Allow up to 3 login attempts
while [ $attempt -le 3 ]; do
    echo "Login attempt $attempt for $user"
    # Prompt for password (hidden input)
    read -s -p "Enter password: " pass
    echo
    # Extract stored password
    stored_pass=$(grep "^$user:" "$PASS_FILE" | cut -d':' -f2)

    if [ "$pass" == "$stored_pass" ]; then
        echo "✅ Access granted!"
        echo "- Switching to user: $user"
        # Simulate switching to user's home directory
        homedir="/home/$user"
        if [ -d "$homedir" ]; then
            cd "$homedir"
            echo "📁 You are now in $homedir"
        else
            echo "⚠️ Home directory not found, staying in current location."
        fi
        echo "📁 Running realistic post-login tasks..."
        # FOR loop replaced: list and count files
        echo "📁 Listing files in home directory:"
        for file in *; do
            echo "  > $file"
        done
    else
        echo "❌ Wrong password"
        attempt=$((attempt + 1))
    fi
done
# If all attempts fail
if [ $attempt -gt 3 ]; then
    echo "❌ Too many failed attempts, access denied."
fi
```

```
echo "📁 Total files: $(ls | wc -l)"
# UNTIL loop replaced: wait until a file named "ready.txt" exists
echo "⌚ Waiting for 'ready.txt' to appear..."
until [ -f "ready.txt" ]; do
    echo "  > File not found, checking again in 2 seconds..."
    sleep 2
done
echo "✅ 'ready.txt' found!"
# WHILE loop: print lines from ready.txt while not empty
echo "📄 Contents of 'ready.txt':"
while read -r line; do
    echo "  > $line"
done < "ready.txt"
# Start a new shell (still as current user, for educational purposes)
exec bash
break
else
    echo "❌ Wrong password"
fi
attempt=$((attempt + 1))
done
# If all attempts fail
if [ $attempt -gt 3 ]; then
    echo "❌ Too many failed attempts, access denied."
fi
```

شرح الscript:

1. تعريف مسار ملف المستخدمين

في البداية نقوم بتخزين مسار ملف users.txt في متغير: PASS_FILE="./users.txt" (الرمز ./ يعني أن الملف موجود في المجلد الحالي الذي يتم تشغيل السكريبت منه)

2. قراءة اسم المستخدم

نطلب من المستخدم إدخال اسم المستخدم، ثم نقوم بتخزينه في المتغير user



3. التحقق من وجود المستخدم:

نقوم بالتحقق مما إذا كان اسم المستخدم موجوداً في ملف users.txt باستخدام الأمر grep

```
# Check if user exists in the file
if ! grep -q "^$user:" "$PASS_FILE"; then
    echo "❌ User '$user' not found in database."
    exit 1
fi
```

حيث:

“^\$user:” يتحقق من أن السطر يبدأ باسم المستخدم متبوعاً بنقطتين.

-q تعني الوضع الصامت، أي لا يعرض أي خرج على ال terminal

! تعني النفي

أي إذا لم يكن اسم المستخدم موجوداً في الملف، يتم إيقاف السكريبت مباشرة باستخدام: exit 1

4. تحديد عدد محاولات تسجيل الدخول

نعرف عداد للمحاولات ونبدأ بالقيمة 1، ثم نستخدم حلقة while تسمح بثلاث محاولات فقط.

5. قراءة كلمة المرور

نطلب من المستخدم إدخال كلمة المرور باستخدام الخيار -s لإخفائها أثناء الكتابة.

6. استخراج كلمة المرور المخزنة

نقوم باستخراج كلمة المرور الخاصة بالمستخدم من الملف users.txt:

```
echo
# Extract stored password
stored_pass=$(grep "^$user:" "$PASS_FILE" | cut -d':' -f2)
```

▪ grep يجلب السطر الخاص بالمستخدم

▪ | يقوم بتمرير خرج grep إلى الأمر التالي

▪ cut -d':' يقسم السطر اعتماداً على الفاصل ":"

(أي إذا كان محتوى السطر مثلاً: aaa:bbb:ccc فتقوم بالتقسيم لـ 3 أعمدة، في مثالنا محتوى السطر

من الشكل: username:password: بالتالي تقوم بالتقسيم لعمودين فقط)

▪ -f2 يعني اختيار العمود الثاني، وهو كلمة المرور

7. مقارنة كلمات المرور والانتقال إلى مجلد المستخدم

إذا كانت كلمة المرور المدخلة مطابقة للمخزنة :

▪ نقوم بتحديد مسار المجلد الأساسي للمستخدم homedir="/home/\$user"

▪ ثم نتحقق من وجوده باستخدام [-d "\$homedir"] ، حيث يرجع الخيار -d قيمة True إذا

كان المجلد موجوداً.

▪ إذا كان موجوداً، ننتقل إليه باستخدام cd ونعرض محتوياته، إذا لم يكن موجوداً نبقى في

المجلد الحالي.

8. عرض الملفات

نستخدم حلقة for لعرض أسماء جميع الملفات في المجلد

```
# For loop: repeated test and code lines
echo "📁 Listing files in home directory:"
for file in *; do
    echo "  > $file"
done
```

- الرمز * يعني جميع الملفات والمجلدات في المسار الحالي

9. الانتظار حتى ظهور ملف ready.txt

```
until [ -f "ready.txt" ]; do
    echo "  > File not found, checking again in 2 seconds..."
    sleep 2
done
```

- f يتحقق من وجود ملف
- until تستمر بالتنفيذ طالما الشرط غير محقق
- Sleep 2 ينتظر ثانيتين بين كل محاولة

يجب أن يكون الملف ready.txt موجوداً في المجلد الحالي، أي إذا قمنا بالإنتقال لمجلد المستخدم مسبقاً يجب أن يكون الملف في مجلد المستخدم لأنه المجلد الحالي.

10. طباعة محتوى الملف سطرًا بسطر:

```
while read -r line; do
    echo "  > $line"
done < "ready.txt"
```

هنا يتم تمرير محتوى الملف إلى الحلقة عبر Standard Input باستخدام < (standard input)

11. شرح الأمر exec bash:

- يقوم الأمر exec باستبدال العملية الحالية (السكربت) بعملية جديدة.
- هنا يتم استبدال السكربت بجلسة Bash جديدة .
- أي أن السكربت ينتهي تماماً ويتم فتح Shell جديد بدلاً منه.
- لا يتم إنشاء عملية فرعية، بل يتم استبدال العملية الحالية.



وفي هذا المثال عمله يكون: بعد تسجيل الدخول بنجاح، يتم إدخال المستخدم مباشرة إلى جلسة Bash جديدة وكأنها جلسة عمل طبيعية.

12. التعامل مع المحاولات الفاشلة:

- إذا كانت كلمة المرور خاطئة: يتم زيادة العداد attempt بمقدار 1 وتستمر الحلقة
- هناك حالتان للخروج من الحلقة:
 1. الخروج باستخدام break عند نجاح تسجيل الدخول
 2. انتهاء عدد المحاولات (أكثر من 3)
- إذا تجاوز عدد المحاولات 3, يتم رفض الوصول وطباعة الرسالة:
"Too many failed attempts, access denied ➡."

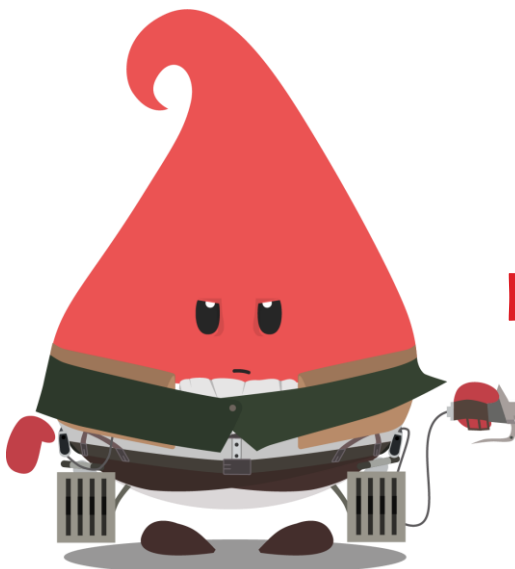
▪ تطبيق السكريبت:

في حال جميع محاولات كلمة المرور خاطئة

```
rbcs@ -VMware-Virtual-Platform:~$ ./loops.sh
Enter username: user1
login attempt 1 for user1
enter password:
wrong pasowrd
login attempt 2 for user1
enter password:
wrong pasowrd
login attempt 3 for user1
enter password:
wrong pasowrd
too many failed attempts, access denied.
```

في حال المستخدم غير موجود

```
rbcs@ -VMware-Virtual-Platform:~$ ./loops.sh
Enter username: user
User 'user' not found in database
```



Don't give up on your dream
Keep Going....

في حال المستخدم موجود وكلمة السر صحيحة، ينتظر حتى يتم إيجاد الملف ready.txt، لذلك نقوم بفتح shell جديدة، ونقوم بالانتقال للمستخدم عن طريق su، وننشئ الملف، نلاحظ أن script توقف وطبع محتوى الملف.

```

rbc@ ~ - VMware-Virtual-Platform: ~ x
user1@ ~ - VMware-Virtual-Platform: ~ x
rbc@ ~ - VMware-Virtual-Platform: ~$ ./loops.sh
Enter username: user1
login attempt 1 for user1
enter password:
Access Granted!
-switching to user: user1
you are now in /home/user1
running realistic post-login tasks...
listing files in home directory:
=> *
total files: 0
waiting for ready.txt to appear...
=> file not found, checking again in 2 seconds
=> file not found, checking again in 2 seconds
=> file not found, checking again in 2 seconds

```

```
rbcs@      -VMware-Virtual-Platform: ~    x    user1@      -VMware-Virtual-Platform: ~    x
rbcs@      -VMware-Virtual-Platform:~$ su user1
Password:
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

user1@      -VMware-Virtual-Platform:/home/rbcs$ cd
user1@      -VMware-Virtual-Platform:~$ ls
user1@      -VMware-Virtual-Platform:~$ vim ready.txt
user1@      -VMware-Virtual-Platform:~$ ls
ready.txt
```

```
rbcsc@      -VMware-Virtual-Platform: ~ x    user1@      -VMware-Virtual-Platform: ~ x v
rbcsc@      -VMware-Virtual-Platform:~$ ./loops.sh
Enter username: user1
login attempt 1 for user1
enter password:
Access Granted!
-switching to user: user1
you are now in /home/user1
running realistic post-login tasks...
listing files in home directory:
=> *
total files: 0
waiting for ready.txt to appear...
=> file not found, checking again in 2 seconds
=> file not found, checking again in 2 seconds
=> file not found, checking again in 2 seconds
=> file not found, checking again in 2 seconds
=> file not found, checking again in 2 seconds
=> file not found, checking again in 2 seconds
=> file not found, checking again in 2 seconds
=> file not found, checking again in 2 seconds
=> file not found, checking again in 2 seconds
=> file not found, checking again in 2 seconds
=> file not found, checking again in 2 seconds
=> file not found, checking again in 2 seconds
=> file not found, checking again in 2 seconds
ready.txt found!
contents of ready.txt:
Hello World!
```

Case Statement

- تستخدم جملة case في Bash لمطابقة الأنماط (Pattern Matching).
- تبدأ بالكلمة المفتاحية case، يليها المتغير المراد اختباره، ثم الكلمة in.
- تنتهي بكلمة esac (وهي كلمة case مكتوبة بالعكس)

بنية جملة case:

```
case $variable in
    pattern1)
        commands
        ;;
    pattern2|pattern3)
        commands
        ;;
    *)
        default_commands
        ;;
esac
```

1. المتغير \$variable: القيمة التي يتم اختبارها ومقارنتها مع الأنماط.
2. الأنماط (Patterns)

- كل فرع (clause) يتكوّن من نمط (Pattern) ومجموعة أوامر تنفذ عند تطابق النمط.
- ينتهي كل فرع بالرمز ;;
- يمكن للأنماط استخدام محارف خاصة مثل:

- * لمطابقة أي سلسلة

- ? لمطابقة حرف واحد

- [...] لمطابقة مجموعة محارف

- الرمز) يُستخدم لإنهاء قائمة الأنماط في الفرع.

- الرمز | يُستخدم للفصل بين عدة أنماط ضمن نفس الفرع.

3. الحالة الافتراضية (Default case)

يُستخدم النمط * كخيار افتراضي في حال عدم تطابق أي نمط آخر.

4. آلية التنفيذ

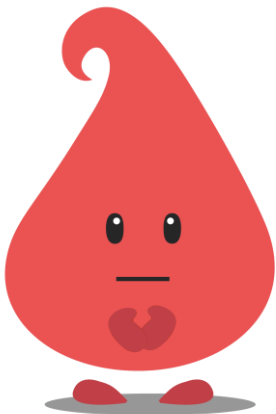
- يتم تنفيذ أوامر أول نمط مطابق فقط.

- بعد التنفيذ، تخرج جملة case مباشرة ولا يتم فحص بقية الأنماط.

5. حالة الخروج (Exit Status)

- إذا لم يتطابق أي نمط: تكون قيمة الخروج 0.

- إذا تطابق أحد الأنماط: تكون قيمة الخروج هي قيمة الخروج للأوامر التي تم تنفيذها.



```
#!/bin/bash
echo "Choose a contact to display information:"
echo "[C]hris Ramsey"
echo "[J]ames Gardner"
echo "[S]arah Snyder"
echo "[R]ose Armstrong"
echo "[A]ll contacts"
echo "[Q]uit"
read person
case "$person" in
    "C" | "c")
        echo "Chris Ramsey"
        echo "cramsey@email.com"
        echo "27 Railroad Dr. Bayside, NY"
        ;;
    "J" | "j")
        echo "James Gardner"
        echo "jgardner@email.com"
        echo "31 Green Street, Green Cove Springs, FL"
        ;;
    "S" | "s")
        echo "Sarah Snyder"
        echo "ssnyder@email.com"
        echo "8059 N. Hartford Court, Syosset, NY"
        ;;
    "R" | "r")
        echo "Rose Armstrong"
        echo "rarmstrong@email.com"
        echo "49 Woodside St., Oak Forest, IL"
        ;;
    "A" | "a")
        echo "Chris Ramsey - cramsey@email.com - 27 Railroad Dr. Bayside, NY"
        echo "James Gardner - jgardner@email.com - 31 Green Street, Green Cove Springs, FL"
        echo "Sarah Snyder - ssnyder@email.com - 8059 N. Hartford Court, Syosset, NY"
        echo "Rose Armstrong - rarmstrong@email.com - 49 Woodside St., Oak Forest, IL"
        ;;
    "Q" | "q")
        echo "Exiting program..."
        exit 0
        ;;
    *)
        echo "Contact doesn't exist."
        ;;
esac
exit 0
```

يقوم هذا السكريبت بإنشاء دفتر عناوين بسيط باستخدام case

- في البداية، يتم عرض قائمة تحتوي على أسماء أشخاص، وكل اسم مرتبط بحرف معين.
- بعد ذلك، يتم قراءة إدخال المستخدم وتخزينه في المتغير person.
- باستخدام جملة case، تتم مقارنة القيمة المدخلة مع مجموعة من الأنماط.
- عند تطابق الإدخال مع أحد الأحرف (سواء كان حرفاً كبيراً أو صغيراً)، يتم طباعة معلومات الشخص المختار. (تم استخدام الرمز | للسماح بقبول الأحرف الكبيرة والصغيرة، مثل: c أو C)

■ تطبيق السكريبت:

```
root@master:/home/mohammad/scripts# ./case.bash
Choose a contact to display information:
[C]hris Ramsey
[J]ames Gardner
[S]arah Snyder
[R]ose Armstrong
[A]ll contacts
[Q]uit
c
Chris Ramsey
cramsey@email.com
27 Railroad Dr. Bayside, NY
```

حلقة Select

تستخدم حلقة select لإنشاء قوائم تفاعلية بسيطة داخل سكريبتات الـ Shell.

بنية جملة select

```
PS3="Prompt text: "
select variable in option1 option2 option3
do
    case $variable in
        option1) commands ;;
        option2) commands ;;
        option3) break ;;
        *) echo "Invalid choice" ;;
    esac
done
```

1. تقوم حلقة select تلقائياً بترقيم عناصر القائمة (options).
2. يقوم المستخدم بإدخال رقم الخيار المطلوب، ويتم تخزين قيمة الخيار المختار في المتغير المحدد داخل حلقة select.
3. المتغير PS3 يُستخدم لتحديد نص رسالة الطلب (Prompt) التي تظهر للمستخدم، قيمته الافتراضية هي: #?.
4. بعد تنفيذ الأوامر الخاصة بالخيار المختار، تعود القائمة للظهور مرة أخرى تلقائياً، ويمكن للمستخدم اختيار خيار جديد دون الحاجة إلى إعادة تشغيل السكريبت.
5. يمكن استخدام الأمر break أو exit للخروج من حلقة select.

مثال تطبيقي 1:

```
#!/bin/bash
PS3="Select a week day (1-7), 8 for all days, or 9 to exit: "

select i in mon tue wed thur fri sat sun "All Days" Exit
do
    case $i in
        mon) echo "Monday";;
        tue) echo "Tuesday";;
        wed) echo "Wednesday";;
        thur) echo "Thursday";;
        fri) echo "Friday";;
        sat) echo "Saturday";;
        sun) echo "Sunday";;
        "All Days") echo "Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday";;
        Exit)
            echo " Goodbye!"
            break
            ;;
        *)
            echo " Invalid choice, please try again."
            ;;
    esac
done
```

- يقوم هذا المثال بإنشاء قائمة تفاعلية باستخدام حلقة select.
- تم تعريف نص رسالة الطلب (Prompt) باستخدام المتغير PS3، حيث يُطلب من المستخدم اختيار رقم اليوم من القائمة.
- تقوم حلقة select بعرض قائمة الأيام مرقمة تلقائياً.
- يُدخل المستخدم رقم الخيار، وليس اسم اليوم، ويتم تخزين القيمة المقابلة لهذا الرقم في المتغير i.
- بعد ذلك، يتم استخدام جملة case لمطابقة القيمة المختارة وتنفيذ الأمر المناسب لها.

تم وضع الخيار "All Days" بين علامتي اقتباس " " حتى يتم التعامل معه كعنصر واحد، وليس ككلمتين منفصلتين.

مثال تطبيقي 2:

```
#!/bin/bash
PS3="Choose an option (1-4): "
select option in \
"List all system user accounts" \
"Show active sessions, unique users, and their names" \
"Show groups of the current user" \
"Exit"
do
    case $option in
        "List all system user accounts")
            lslogins -o USER
            ;;
        "Show active sessions, unique users, and their names")
            echo "Active sessions: $(who | wc -l)"
            echo "Unique logged-in users: $(who | awk '{print $1}' | sort -u | wc -l)"
            echo "Logged-in usernames:"
            who | awk '{print $1}' | sort -u
            ;;
        "Show groups of the current user")
            groups
            ;;
        "Exit")
            echo " Goodbye!"
            break
            ;;
        *)
            echo " Invalid choice, please try again."
            ;;
    esac
done
```

- استخدام \ في جملة select: يُستخدم لمتابعة السطر، أي أن الأمر مستمر في السطر التالي، فهو لا يغير من عمل الكود إطلاقاً، فقط لتحسين التنسيق وتسهيل القراءة.
- "Exit" بما أنها كلمة واحدة، لا يشترط وضعها بين علامتي اقتباس " "، لكن إن وُضعت لن يؤثر ذلك على عمل السكريبت.



شرح كل خيار من case مع تنفيذ:

1. عرض جميع حسابات المستخدمين في النظام

```
rbcs@ -VMware-Virtual-Platform:~$ lslogins -o USER
USER
root
daemon
bin
sys
sync
games
man
lp
mail
news
uucp
proxy
www-data
backup
list
```

2. عرض الجلسات والمستخدمين النشطين

- who: يقوم بعرض الجلسات النشطة حالياً على النظام، أي المستخدمين الذين قاموا بتسجيل الدخول.
- wc -l: يقوم بعدد الأسطر في الخرج (حيث أن wc اختصار لـ word count، والخيار -l يعني line count)
- awk '{print \$1}': تأخذ الخرج السابق و تطبع العمود الأول \$1
- sort -u: يقوم بترتيب النتائج وإزالة القيم المكررة للحصول على أسماء مستخدمين فريدة فقط (unique users)

توضيح لنتيجة كل تعليمة على حدا:

```
rbcs@ -VMware-Virtual-Platform:~$ who
rbcs    seat0      2025-12-22 22:32 (login screen)
rbcs    tty2      2025-12-22 22:32 (tty2)
rbcs@ -VMware-Virtual-Platform:~$ who | wc -l
2
rbcs@ -VMware-Virtual-Platform:~$ who | awk '{print $1}'
rbcs
rbcs
rbcs@ -VMware-Virtual-Platform:~$ who | awk '{print $2}'
seat0
tty2
rbcs@ -VMware-Virtual-Platform:~$ who | awk '{print $1}' | sort -u
rbcs
rbcs@ -VMware-Virtual-Platform:~$ who | awk '{print $1}' | sort -u | wc -l
1
rbcs@ -VMware-Virtual-Platform:~$
```

أكبر إنجاز يمكنك تحقيقه اليوم

هو أن تكون أفضل من أمس!